Table of Contents

..... 1
Mission constants. 1
Vehicle data. 2
Parameters passed to the local functions. 2
Initial heading corresponding to the target launch azimuth. 3
Inertial insertion state. 3
Deployed orbital state for the remaining calculations. 4
Plane change into the target polar GEO. 5
GEO phasing maneuver for rendezvous. 5
Orbital elements from actual post-burn ECI states. 7
Required command window output. 8
Launch plots. 10
2D GTO trajectory in the GTO perifocal frame. 15
2D phasing orbit in the final GEO orbital plane. 16
3D trajectory from deployment to rendezvous. 17
Local functions. 19
Discussion of the design reference mission. 28
AI Use Statement. 29


```
clc;
clear;
close all;

% AEM369 Midterm Project
% Design Reference Mission
% Collin Atkins, 3/25/2026
```

Mission constants.

```
RE = 6378e3;
MU = 3.986e14;
G0 = 9.807;
OMEGA_EARTH = 7.292e-5;

RHO0 = 1.225;
H_SCALE = 7.5e3;

launch_latitude = deg2rad(28.5744);
launch_longitude = deg2rad(-80.6520);
launch_altitude = 3.0;

launch_azimuth_target = deg2rad(35.0);

pitchover_altitude = 100.0;
pitchover_flight_path_angle = deg2rad(85.0);

target_raan = deg2rad(87.57);
target_inclination = deg2rad(-90.0);
```

```
target_argument_of_periapsis = deg2rad(0.0);
target_eccentricity = 0.0;
target_semi_major_axis = 42164e3;
target_true_anomaly_epoch = deg2rad(38.0);
```

Vehicle data.

```
structural_ratio = 0.10;

stage_1_thrust = 1500e3;
stage_1_isp = 310.0;
stage_1_propellant_mass = 50000.0;

stage_2_thrust = 200e3;
stage_2_isp = 350.0;
stage_2_propellant_mass = 12000.0;

reference_area = 20.0;
drag_coef = 0.5;

chaser_empty_mass = 350.0;
chaser_total_mass = 3500.0;
chaser_isp = 350.0;

stage_1_structural_mass = ...
    structural_ratio / (1.0 - structural_ratio) * stage_1_propellant_mass;

stage_2_structural_mass = ...
    structural_ratio / (1.0 - structural_ratio) * stage_2_propellant_mass;

initial_total_mass = ...
    stage_1_propellant_mass + stage_1_structural_mass + ...
    stage_2_propellant_mass + stage_2_structural_mass + ...
    chaser_total_mass;
```

Parameters passed to the local functions.

```
mission.RE = RE;
mission.MU = MU;
mission.G0 = G0;
mission.OMEGA_EARTH = OMEGA_EARTH;
mission.RHO0 = RHO0;
mission.H_SCALE = H_SCALE;
mission.launch_latitude = launch_latitude;
mission.launch_longitude = launch_longitude;
mission.launch_altitude = launch_altitude;
mission.launch_azimuth_target = launch_azimuth_target;
mission.pitchover_altitude = pitchover_altitude;
mission.pitchover_flight_path_angle = pitchover_flight_path_angle;
mission.target_raan = target_raan;
mission.target_inclination = target_inclination;
mission.target_argument_of_periapsis = target_argument_of_periapsis;
mission.target_eccentricity = target_eccentricity;
```

```
mission.target_semi_major_axis = target_semi_major_axis;
mission.target_true_anomaly_epoch = target_true_anomaly_epoch;

vehicle.reference_area = reference_area;
vehicle.drag_coef = drag_coef;
vehicle.stage_1_thrust = stage_1_thrust;
vehicle.stage_1_isp = stage_1_isp;
vehicle.stage_1_propellant_mass = stage_1_propellant_mass;
vehicle.stage_1_structural_mass = stage_1_structural_mass;
vehicle.stage_2_thrust = stage_2_thrust;
vehicle.stage_2_isp = stage_2_isp;
vehicle.stage_2_propellant_mass = stage_2_propellant_mass;
vehicle.stage_2_structural_mass = stage_2_structural_mass;
vehicle.chaser_empty_mass = chaser_empty_mass;
vehicle.chaser_total_mass = chaser_total_mass;
vehicle.chaser_isp = chaser_isp;
vehicle.initial_total_mass = initial_total_mass;
```

Initial heading corresponding to the target launch azimuth.

```
heading_cost = @(initial_heading) ...
    launch_heading_cost(initial_heading, mission, vehicle);

initial_heading_guess = ...
    fminbnd(heading_cost, deg2rad(20.0), deg2rad(160.0));

launch = simulate_launch(initial_heading_guess, mission, vehicle);
```

Inertial insertion state.

```
time_launch = launch.time(end);

north_hat = local_north_hat(launch.insertion.latitude, ...
    launch.insertion.longitude);

east_hat = local_east_hat(launch.insertion.longitude);

up_hat = local_up_hat(launch.insertion.latitude, ...
    launch.insertion.longitude);

position_ecef = launch.insertion.radius * up_hat;

velocity_ground_ecef = ...
    launch.insertion.airspeed * ...
    (cos(launch.insertion.flight_path_angle) * ...
    (cos(launch.insertion.heading) * north_hat + ...
    sin(launch.insertion.heading) * east_hat) + ...
    sin(launch.insertion.flight_path_angle) * up_hat);

earth_rotation_velocity_ecef = ...
```

```
    cross([0.0; 0.0; OMEGA_EARTH], position_ecef);

theta_earth = OMEGA_EARTH * time_launch;

ecef_to_eci = [ ...
    cos(theta_earth), -sin(theta_earth), 0.0; ...
    sin(theta_earth),  cos(theta_earth), 0.0; ...
    0.0,               0.0,              1.0];

north_hat_eci = ecef_to_eci * north_hat;
east_hat_eci = ecef_to_eci * east_hat;
up_hat_eci = ecef_to_eci * up_hat;

position_inertial = ecef_to_eci * position_ecef;
velocity_inertial = ...
    ecef_to_eci * (velocity_ground_ecef + ...
    earth_rotation_velocity_ecef);

velocity_horizontal = ...
    velocity_inertial - ...
    dot(velocity_inertial, up_hat_eci) * up_hat_eci;

launch_azimuth = atan2( ...
    dot(velocity_horizontal, east_hat_eci), ...
    dot(velocity_horizontal, north_hat_eci));

velocity_inertial_mag = norm(velocity_inertial);
```

Deployed orbital state for the remaining calculations.

The actual deployed inertial position and velocity are used as the starting point for the subsequent orbital calculations.

```
insertion_radius = norm(position_inertial);
insertion_speed = norm(velocity_inertial);
insertion_coe = rv_to_coe(position_inertial, velocity_inertial, MU);

r_perigee = insertion_radius;
r_apogee = target_semi_major_axis;
a_gto = 0.5 * (r_perigee + r_apogee);
e_gto = (r_apogee - r_perigee) / (r_apogee + r_perigee);
p_gto = a_gto * (1.0 - e_gto^2);

speed_gto_perigee = sqrt(MU * (2.0 / r_perigee - 1.0 / a_gto));
speed_geo = sqrt(MU / r_apogee);
speed_gto_apogee = sqrt(MU * (2.0 / r_apogee - 1.0 / a_gto));

delta_v_1 = speed_gto_perigee - insertion_speed;
delta_v_2 = speed_geo - speed_gto_apogee;
time_gto = pi * sqrt(a_gto^3 / MU);
```

```
gto_coe.a = a_gto;
gto_coe.e = e_gto;
gto_coe.i = insertion_coe.i;
gto_coe.raan = insertion_coe.raan;
gto_coe.omega = 0.0;
gto_coe.nu = 0.0;

initial_geo_coe.a = target_semi_major_axis;
initial_geo_coe.e = 0.0;
initial_geo_coe.i = insertion_coe.i;
initial_geo_coe.raan = insertion_coe.raan;
initial_geo_coe.omega = 0.0;
initial_geo_coe.nu = 0.0;
```

Plane change into the target polar GEO.

```
h_hat_initial = orbital_plane_normal( ...
    initial_geo_coe.raan, initial_geo_coe.i);
h_hat_target = orbital_plane_normal( ...
    target_raan, target_inclination);

plane_change_angle = acos(max(-1.0, min(1.0, abs(dot(h_hat_initial, ...
    h_hat_target)))));

delta_v_3 = 2.0 * speed_geo * sin(0.5 * plane_change_angle);
```

GEO phasing maneuver for rendezvous.

First coast on the initial GEO to a true node with the target plane.

```
mean_motion_geo = sqrt(MU / target_semi_major_axis^3);

geo_burn_hat = -position_inertial / norm(position_inertial);

node_line_hat = cross(h_hat_initial, h_hat_target);
node_line_hat = node_line_hat / norm(node_line_hat);

angle_to_node_1 = wrap_to_2pi(atan2( ...
    dot(cross(geo_burn_hat, node_line_hat), h_hat_initial), ...
    dot(geo_burn_hat, node_line_hat)));

angle_to_node_2 = wrap_to_2pi(atan2( ...
    dot(cross(geo_burn_hat, -node_line_hat), h_hat_initial), ...
    dot(geo_burn_hat, -node_line_hat)));

if angle_to_node_1 <= angle_to_node_2
    node_hat = node_line_hat;
    angle_geo_coast = angle_to_node_1;
else
    node_hat = -node_line_hat;
    angle_geo_coast = angle_to_node_2;
end
```

```
time_geo_coast = angle_geo_coast / mean_motion_geo;
plane_change_point = target_semi_major_axis * node_hat;

% Target anomaly at the node point in the target polar GEO plane.
p_hat_target_orbit = [cos(target_raan); sin(target_raan); 0.0];
q_hat_target_orbit = [ ...
    -sin(target_raan) * cos(target_inclination); ...
     cos(target_raan) * cos(target_inclination); ...
     sin(target_inclination)];

target_true_anomaly_node = wrap_to_2pi(atan2( ...
    dot(node_hat, q_hat_target_orbit), ...
    dot(node_hat, p_hat_target_orbit)));

time_before_phasing = time_launch + time_gto + time_geo_coast;

target_true_anomaly_at_phasing_start = wrap_to_2pi( ...
    target_true_anomaly_epoch + mean_motion_geo * time_before_phasing);

time_phase = wrap_to_2pi( ...
    target_true_anomaly_node - target_true_anomaly_at_phasing_start) / ...
    mean_motion_geo;

if time_phase < 1.0
    time_phase = time_phase + 2.0 * pi / mean_motion_geo;
end

a_phase = (MU * (time_phase / (2.0 * pi))^2)^(1.0 / 3.0);

if a_phase < 0.5 * target_semi_major_axis
    time_phase = time_phase + 2.0 * pi / mean_motion_geo;
    a_phase = (MU * (time_phase / (2.0 * pi))^2)^(1.0 / 3.0);
end

if a_phase >= target_semi_major_axis
    phase_e = 1.0 - target_semi_major_axis / a_phase;
    phase_omega = 0.0;
    phase_nu_burn = 0.0;
else
    phase_e = target_semi_major_axis / a_phase - 1.0;
    phase_omega = pi;
    phase_nu_burn = pi;
end

speed_phase_at_burn = ...
    sqrt(MU * (2.0 / target_semi_major_axis - 1.0 / a_phase));

delta_v_4 = abs(speed_phase_at_burn - speed_geo);
delta_v_5 = delta_v_4;

phasing_coe.a = a_phase;
phasing_coe.e = phase_e;
phasing_coe.i = target_inclination;
phasing_coe.raan = target_raan;
```

```
phasing_coe.omega = phase_omega;
phasing_coe.nu = phase_nu_burn;

total_delta_v = ...
    delta_v_1 + delta_v_2 + delta_v_3 + delta_v_4 + delta_v_5;

total_time = time_launch + time_gto + time_geo_coast + time_phase;
```

Orbital elements from actual post-burn ECI states.

```
position_gto = position_inertial;
velocity_gto = velocity_inertial + ...
    delta_v_1 * velocity_inertial / norm(velocity_inertial);

gto_coe_report = rv_to_coe(position_gto, velocity_gto, MU);

r_hat_perigee = position_gto / norm(position_gto);

h_hat_gto_actual = cross(position_gto, velocity_gto);
h_hat_gto_actual = h_hat_gto_actual / norm(h_hat_gto_actual);

t_hat_perigee = cross(h_hat_gto_actual, r_hat_perigee);
t_hat_perigee = t_hat_perigee / norm(t_hat_perigee);

if dot(t_hat_perigee, velocity_gto) < 0.0
    t_hat_perigee = -t_hat_perigee;
end

position_geo_burn = -target_semi_major_axis * r_hat_perigee;
velocity_geo_plus = speed_geo * (-t_hat_perigee);

initial_geo_coe_report = ...
    rv_to_coe(position_geo_burn, velocity_geo_plus, MU);

if initial_geo_coe_report.e < 1.0e-8
    initial_geo_coe_report.e = 0.0;
    initial_geo_coe_report.omega = 0.0;
    initial_geo_coe_report.nu = 0.0;
end

u_hat_geo = position_geo_burn / norm(position_geo_burn);
v_hat_geo = cross(h_hat_initial, u_hat_geo);
v_hat_geo = v_hat_geo / norm(v_hat_geo);

t_hat_initial_node = ...
    -u_hat_geo * sin(angle_geo_coast) + ...
    v_hat_geo * cos(angle_geo_coast);
t_hat_initial_node = t_hat_initial_node / norm(t_hat_initial_node);

burn_hat = plane_change_point / norm(plane_change_point);
t_hat_target_node = cross(h_hat_target, burn_hat);
```

```
t_hat_target_node = t_hat_target_node / norm(t_hat_target_node);

if dot(t_hat_target_node, t_hat_initial_node) < 0.0
    t_hat_target_node = -t_hat_target_node;
end

position_phase_start = plane_change_point;
velocity_phase_start = speed_phase_at_burn * t_hat_target_node;

phasing_coe_report = rv_to_coe( ...
    position_phase_start, velocity_phase_start, MU);
```

Required command window output.

```
fprintf('\n--- PITCHOVER ---\n');
fprintf('Initial flight-path angle = %.2f deg\n', ...
    rad2deg(pitchover_flight_path_angle));

fprintf('\n--- MASSES ---\n');
fprintf('Stage 1 propellant = %.2f kg\n', stage_1_propellant_mass);
fprintf('Stage 2 propellant = %.2f kg\n', stage_2_propellant_mass);
fprintf('Chaser structural mass = %.2f kg\n', chaser_empty_mass);

fprintf('\n--- INSERTION ---\n');
fprintf('Altitude = %.2f km\n', ...
    (launch.insertion.radius - RE) / 1000.0);
fprintf('Latitude = %.2f deg\n', rad2deg(launch.insertion.latitude));
fprintf('Longitude = %.2f deg\n', rad2deg(launch.insertion.longitude));
fprintf('Launch azimuth = %.2f deg\n', rad2deg(launch_azimuth));
fprintf('Heading = %.2f deg\n', rad2deg(launch.insertion.heading));
fprintf(['Inertial velocity = [%.2f, %.2f, %.2f] m/s ' ...
    'in ECI\n'], ...
    velocity_inertial(1), ...
    velocity_inertial(2), ...
    velocity_inertial(3));
fprintf('Inertial velocity magnitude = %.2f m/s\n', ...
    velocity_inertial_mag);

fprintf('\n--- ORBIT ELEMENTS ---\n');

fprintf('GTO: a = %.2f km, e = %.6f, i = %.2f deg\n', ...
    gto_coe_report.a / 1000.0, ...
    gto_coe_report.e, ...
    rad2deg(gto_coe_report.i));
fprintf(' RAAN = %.2f deg, omega = %.2f deg, nu = %.2f deg\n', ...
    rad2deg(gto_coe_report.raan), ...
    rad2deg(gto_coe_report.omega), ...
    rad2deg(gto_coe_report.nu));

fprintf('Initial GEO: a = %.2f km, e = %.6f, i = %.2f deg\n', ...
    initial_geo_coe_report.a / 1000.0, ...
    initial_geo_coe_report.e, ...
    rad2deg(initial_geo_coe_report.i));
```

```
fprintf([' RAAN = %.2f deg, omega = %.2f deg, ' ...
    'nu = %.2f deg\n'], ...
    rad2deg(initial_geo_coe_report.raan), ...
    rad2deg(initial_geo_coe_report.omega), ...
    rad2deg(initial_geo_coe_report.nu));

fprintf('Phasing: a = %.2f km, e = %.6f, i = %.2f deg\n', ...
    phasing_coe_report.a / 1000.0, ...
    phasing_coe_report.e, ...
    rad2deg(phasing_coe_report.i));
fprintf(' RAAN = %.2f deg, omega = %.2f deg, nu = %.2f deg\n', ...
    rad2deg(phasing_coe_report.raan), ...
    rad2deg(phasing_coe_report.omega), ...
    rad2deg(phasing_coe_report.nu));

fprintf('\n--- DELTA-V ---\n');
fprintf('Maneuver 1: insertion-to-GTO burn = %.2f m/s\n', delta_v_1);
fprintf('Maneuver 2: GTO circularization = %.2f m/s\n', delta_v_2);
fprintf('Maneuver 3: plane change = %.2f m/s\n', delta_v_3);
fprintf('Maneuver 4: phasing burn 1 = %.2f m/s\n', delta_v_4);
fprintf('Maneuver 5: phasing burn 2 = %.2f m/s\n', delta_v_5);
fprintf('Total delta-v = %.2f m/s\n', total_delta_v);

fprintf('\n--- TIME OF FLIGHT ---\n');
fprintf('Launch = %.2f s\n', time_launch);
fprintf('GTO = %.2f s\n', time_gto);
fprintf('Initial GEO coast = %.2f s\n', time_geo_coast);
fprintf('Plane change = 0.00 s\n');
fprintf('Phasing = %.2f s\n', time_phase);
fprintf('Total = %.2f s\n', total_time);
```

```
--- PITCHOVER ---
Initial flight-path angle = 85.00 deg

--- MASSES ---
Stage 1 propellant = 50000.00 kg
Stage 2 propellant = 12000.00 kg
Chaser structural mass = 350.00 kg

--- INSERTION ---
Altitude = 218.11 km
Latitude = 39.64 deg
Longitude = -72.97 deg
Launch azimuth = 35.00 deg
Heading = 32.12 deg
Inertial velocity = [2342.96, 4245.61, 3942.17] m/s in ECI
Inertial velocity magnitude = 6249.43 m/s

--- ORBIT ELEMENTS ---
GTO: a = 24380.05 km, e = 0.729447, i = 63.79 deg
 RAAN = 264.70 deg, omega = 45.32 deg, nu = 0.00 deg
Initial GEO: a = 42164.00 km, e = 0.000000, i = 63.79 deg
 RAAN = 264.70 deg, omega = 0.00 deg, nu = 0.00 deg
```

```
Phasing: a = 36147.89 km, e = 0.166430, i = 90.00 deg
 RAAN = 267.57 deg, omega = 185.81 deg, nu = 180.00 deg

--- DELTA-V ---
Maneuver 1: insertion-to-GTO burn = 3973.57 m/s
Maneuver 2: GTO circularization = 1475.39 m/s
Maneuver 3: plane change = 1401.98 m/s
Maneuver 4: phasing burn 1 = 267.49 m/s
Maneuver 5: phasing burn 2 = 267.49 m/s
Total delta-v = 7385.92 m/s

--- TIME OF FLIGHT ---
Launch = 417.32 s
GTO = 18942.32 s
Initial GEO coast = 33783.69 s
Plane change = 0.00 s
Phasing = 68396.79 s
Total = 121540.13 s
```

Launch plots.

```
figure;
plot(launch.time, launch.airspeed, 'LineWidth', 1.5);
grid on;
xlabel('Time since lift-off (s)');
ylabel('Airspeed (m/s)');
title('Launch Airspeed vs. Time');

drawnow;
snapnow;

figure;
plot(launch.time, rad2deg(launch.flight_path_angle), 'LineWidth', 1.5);
grid on;
xlabel('Time since lift-off (s)');
ylabel('Flight-path angle (deg)');
title('Launch Flight-Path Angle vs. Time');

drawnow;
snapnow;

figure;
plot(launch.time, rad2deg(launch.heading), 'LineWidth', 1.5);
grid on;
xlabel('Time since lift-off (s)');
ylabel('Heading angle (deg)');
title('Launch Heading Angle vs. Time');

drawnow;
snapnow;

figure;
plot(launch.time, rad2deg(launch.latitude), 'LineWidth', 1.5);
```

```
grid on;
xlabel('Time since lift-off (s)');
ylabel('Latitude (deg)');
title('Launch Latitude vs. Time');

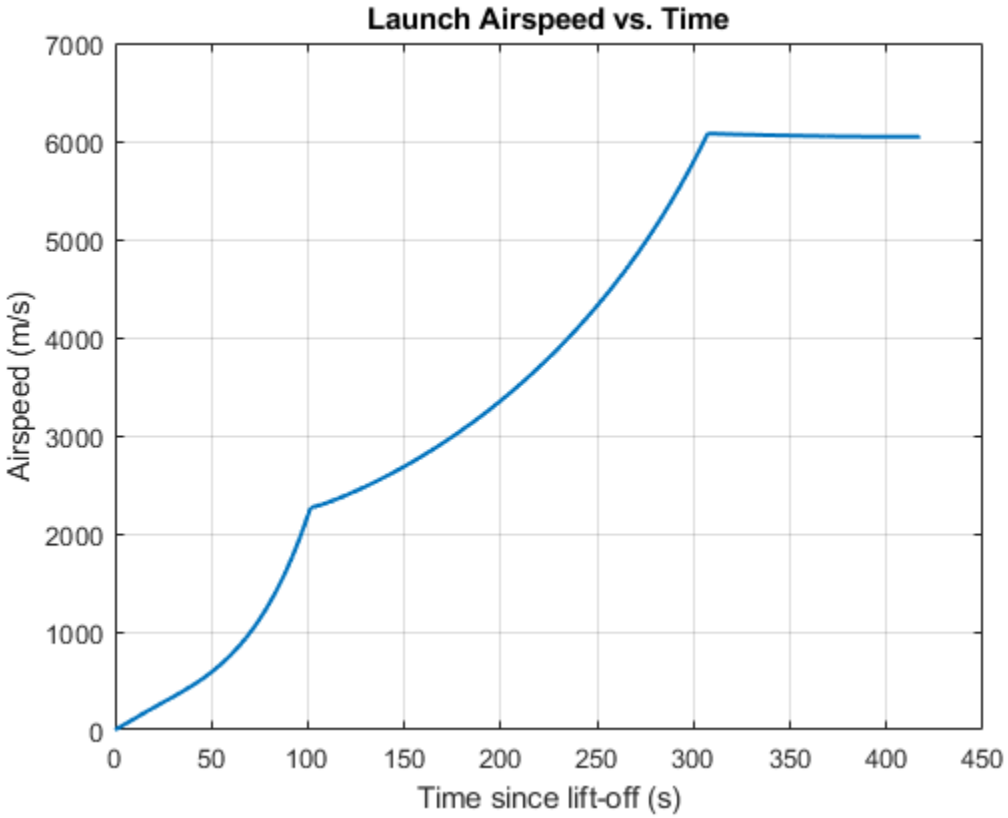
drawnow;
snapnow;

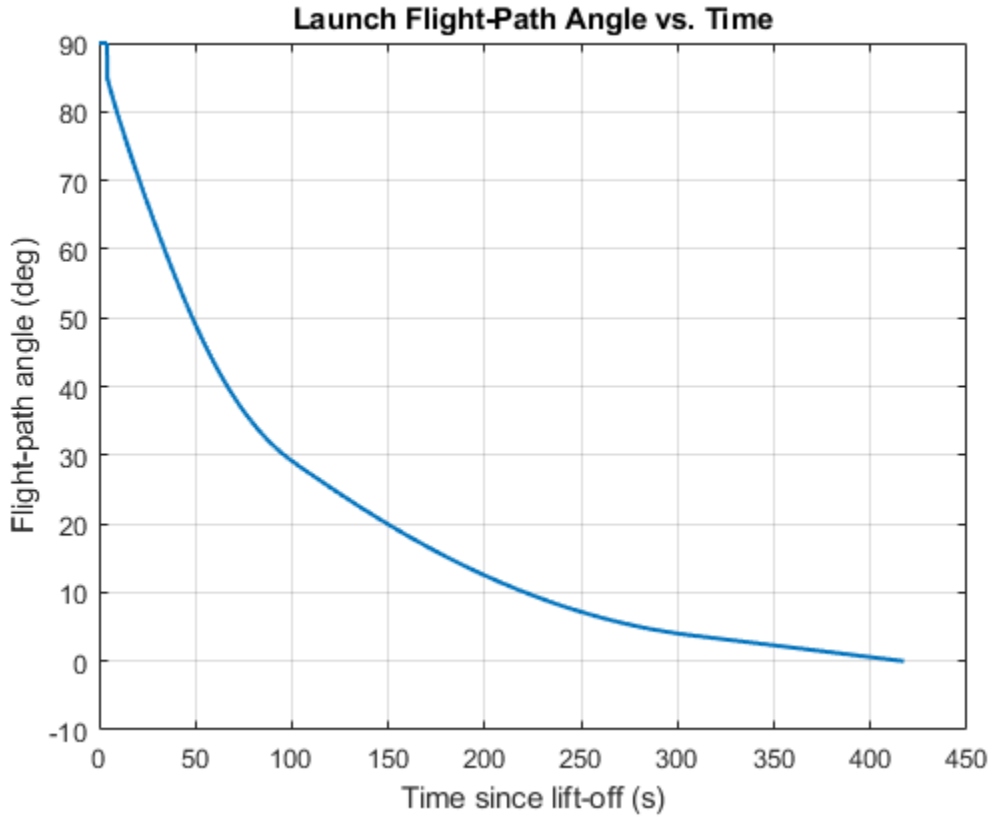
figure;
plot(launch.time, rad2deg(launch.longitude), 'LineWidth', 1.5);
grid on;
xlabel('Time since lift-off (s)');
ylabel('Longitude (deg)');
title('Launch Longitude vs. Time');

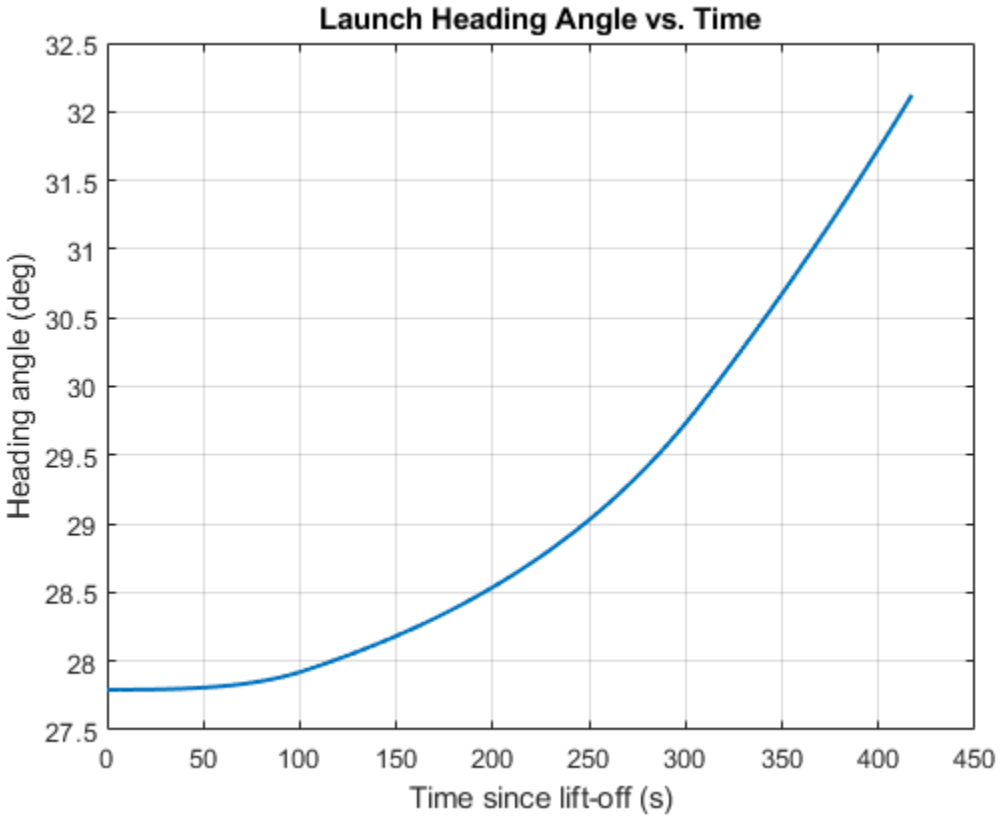
drawnow;
snapnow;

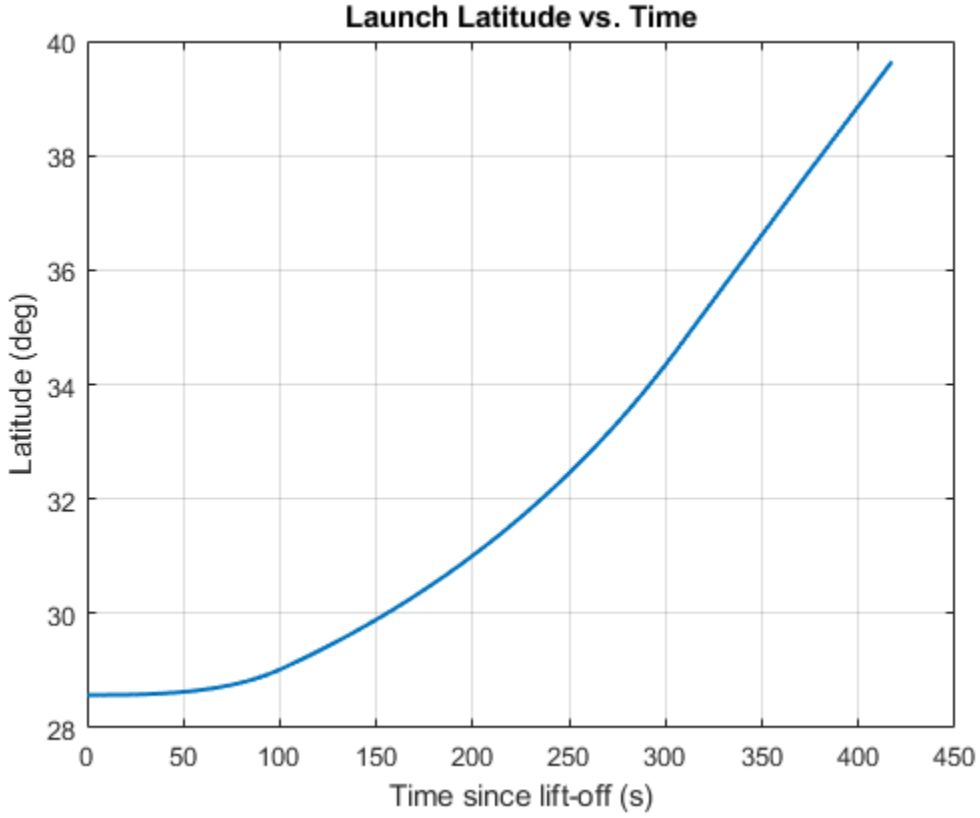
figure;
plot(launch.time, launch.altitude / 1000.0, 'LineWidth', 1.5);
grid on;
xlabel('Time since lift-off (s)');
ylabel('Altitude (km)');
title('Launch Altitude vs. Time');

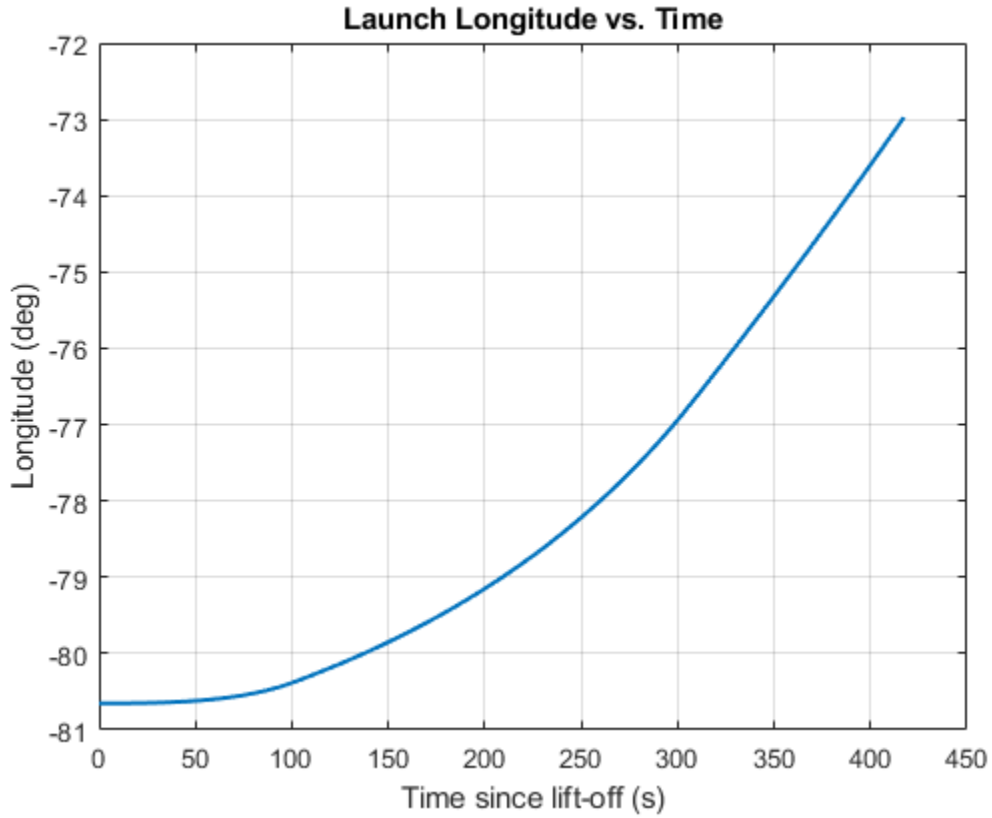
drawnow;
snapnow;
```

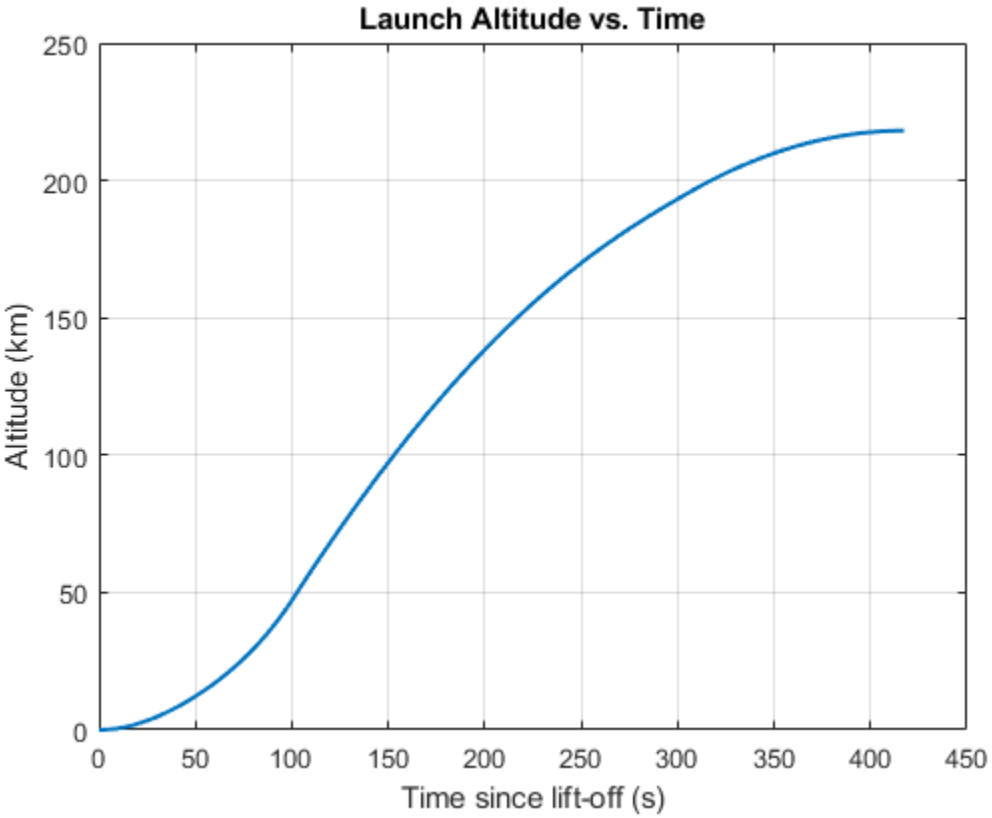

Launch Airspeed vs. Time
Airspeed (m/s)
Time since lift-off (s)
0
1000
2000
3000
4000
5000
6000
7000
0
50
100
150
200
250
300
350
400
450


Launch Flight-Path Angle vs. Time
Flight-path angle (deg)
Time since lift-off (s)
-10
0
10
20
30
40
50
60
70
80
90
0
50
100
150
200
250
300
350
400
450


Launch Heading Angle vs. Time
Heading angle (deg)
32.5
32
31.5
31
30.5
30
29.5
29
28.5
28
27.5
0
50
100
150
200
250
300
350
400
450
Time since lift-off (s)


Launch Latitude vs. Time
Latitude (deg)
40
38
36
34
32
30
28
0
50
100
150
200
250
300
350
400
450
Time since lift-off (s)


Launch Longitude vs. Time
Longitude (deg)
-72
-73
-74
-75
-76
-77
-78
-79
-80
-81
0
50
100
150
200
250
300
350
400
450
Time since lift-off (s)


Launch Altitude vs. Time
Altitude (km)
250
200
150
100
50
0
0
50
100
150
200
250
300
350
400
450
Time since lift-off (s)

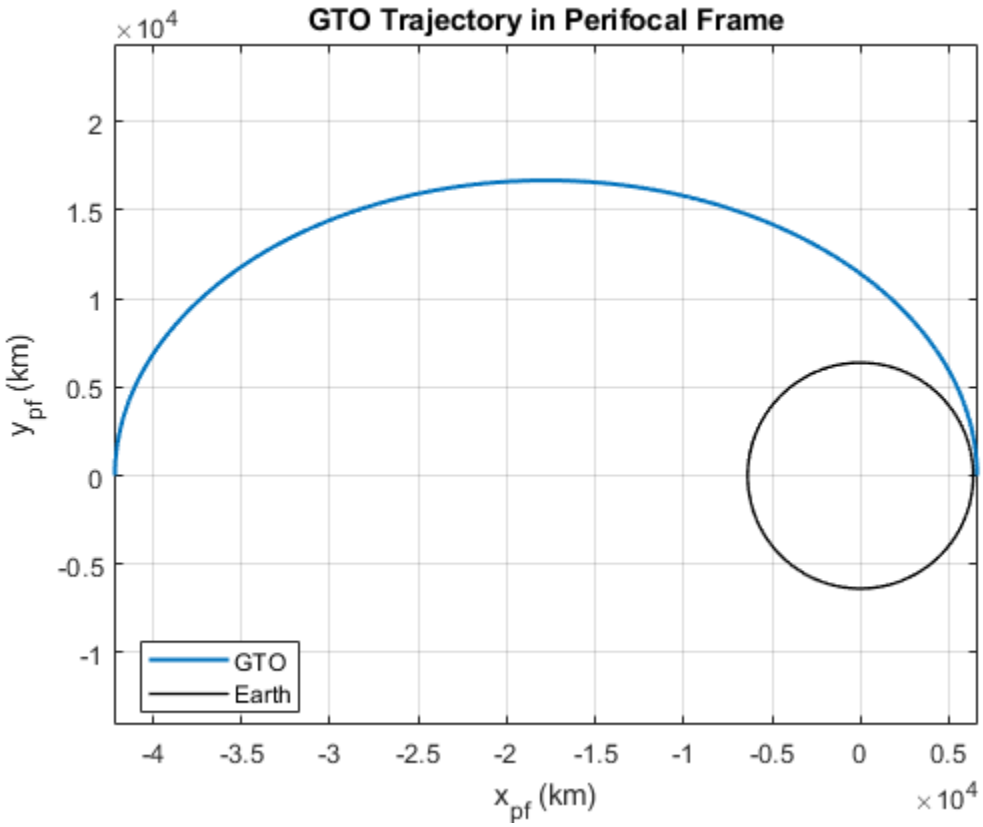
2D GTO trajectory in the GTO perifocal frame.

```
nu_gto = linspace(0.0, pi, 500);
r_gto = p_gto ./ (1.0 + e_gto * cos(nu_gto));
x_gto = r_gto .* cos(nu_gto);
y_gto = r_gto .* sin(nu_gto);

theta_earth = linspace(0.0, 2.0 * pi, 400);
x_earth = RE * cos(theta_earth);
y_earth = RE * sin(theta_earth);

figure;
plot(x_gto / 1000.0, y_gto / 1000.0, 'LineWidth', 1.5);
hold on;
plot(x_earth / 1000.0, y_earth / 1000.0, 'k', 'LineWidth', 1.0);
axis equal;
grid on;
xlabel('x_{pf} (km)');
ylabel('y_{pf} (km)');
title('GTO Trajectory in Perifocal Frame');
legend('GTO', 'Earth', 'Location', 'best');

drawnow;
snapnow;
```



2D phasing orbit in the final GEO orbital plane.

The burn point is placed at the x-intercept.

```
p_phase = phasing_coe.a * (1.0 - phasing_coe.e^2);

r_perigee_phase = phasing_coe.a * (1.0 - phasing_coe.e);
r_apogee_phase  = phasing_coe.a * (1.0 + phasing_coe.e);

burn_r = p_phase / (1.0 + phasing_coe.e * cos(phasing_coe.nu));
r_burn = burn_r;

burn_theta = phasing_coe.nu + phasing_coe.omega;

p_phase = phasing_coe.a * (1.0 - phasing_coe.e^2);

r_perigee_phase = phasing_coe.a * (1.0 - phasing_coe.e);
r_apogee_phase = phasing_coe.a * (1.0 + phasing_coe.e);

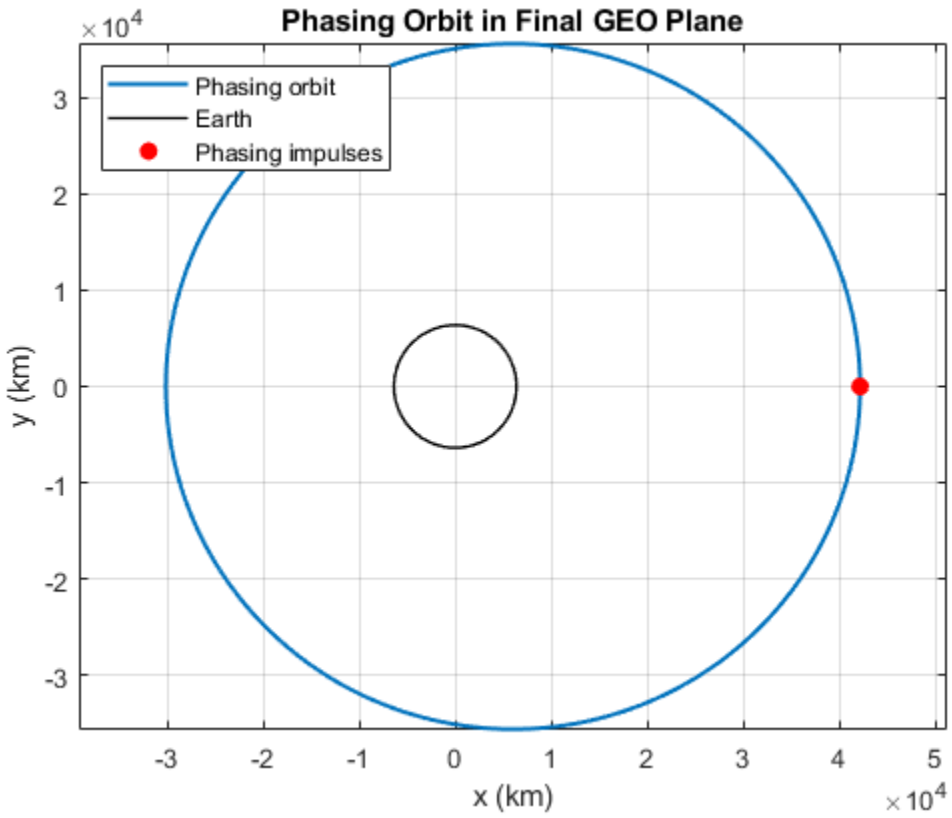
nu_phase = linspace(phasing_coe.nu, phasing_coe.nu + 2.0 * pi, 800);
r_phase = p_phase ./ (1.0 + phasing_coe.e * cos(nu_phase));

theta_phase = nu_phase + phasing_coe.omega;
x_phase = r_phase .* cos(theta_phase);
y_phase = r_phase .* sin(theta_phase);

burn_r = r_phase(1);
burn_x = x_phase(1);
burn_y = y_phase(1);

figure;
plot(x_phase / 1000.0, y_phase / 1000.0, 'LineWidth', 1.5);
hold on;
plot(x_earth / 1000.0, y_earth / 1000.0, 'k', 'LineWidth', 1.0);
plot(burn_x / 1000.0, burn_y / 1000.0, 'ro', 'MarkerFaceColor', 'r');
axis equal;
grid on;
xlabel('x (km)');
ylabel('y (km)');
title('Phasing Orbit in Final GEO Plane');
legend('Phasing orbit', 'Earth', 'Phasing impulses', ...
    'Location', 'northwest');

drawnow;
snapnow;
```



3D trajectory from deployment to rendezvous.

The trajectory is shown in the geocentric celestial frame.

```
p_hat_gto = position_inertial / norm(position_inertial);

h_hat_gto = cross(position_inertial, velocity_inertial);
h_hat_gto = h_hat_gto / norm(h_hat_gto);

q_hat_gto = cross(h_hat_gto, p_hat_gto);
q_hat_gto = q_hat_gto / norm(q_hat_gto);

gto_positions = ...
    p_hat_gto * x_gto + ...
    q_hat_gto * y_gto;

geo_burn_point = gto_positions(:, end);

% Initial GEO coast from Burn 2 to the node where the plane change occurs.
u_hat_geo = geo_burn_point / norm(geo_burn_point);

v_hat_geo = cross(h_hat_initial, u_hat_geo);
v_hat_geo = v_hat_geo / norm(v_hat_geo);

theta_geo_coast = linspace(0.0, angle_geo_coast, 250);
```

```
geo_coast_positions = target_semi_major_axis * ...
    (u_hat_geo * cos(theta_geo_coast) + ...
     v_hat_geo * sin(theta_geo_coast));

burn_hat = plane_change_point / norm(plane_change_point);

t_hat = cross(h_hat_target, burn_hat);
t_hat = t_hat / norm(t_hat);

phase_positions = ...
    burn_hat * x_phase + ...
    t_hat * y_phase;

phase_burn_point = phase_positions(:, 1);
rendezvous_point = phase_positions(:, end);

figure;
plot3(gto_positions(1, :) / 1000.0, ...
    gto_positions(2, :) / 1000.0, ...
    gto_positions(3, :) / 1000.0, 'LineWidth', 1.5);
hold on;

plot3(geo_coast_positions(1, :) / 1000.0, ...
    geo_coast_positions(2, :) / 1000.0, ...
    geo_coast_positions(3, :) / 1000.0, 'LineWidth', 1.5);

plot3(phase_positions(1, :) / 1000.0, ...
    phase_positions(2, :) / 1000.0, ...
    phase_positions(3, :) / 1000.0, 'LineWidth', 1.5);

plot3(gto_positions(1, 1) / 1000.0, ...
    gto_positions(2, 1) / 1000.0, ...
    gto_positions(3, 1) / 1000.0, ...
    'ro', 'MarkerFaceColor', 'r');

plot3(geo_burn_point(1) / 1000.0, ...
    geo_burn_point(2) / 1000.0, ...
    geo_burn_point(3) / 1000.0, ...
    'ks', 'MarkerFaceColor', 'k');

plot3(phase_burn_point(1) / 1000.0, ...
    phase_burn_point(2) / 1000.0, ...
    phase_burn_point(3) / 1000.0, ...
    'kd', 'MarkerFaceColor', 'k', 'MarkerSize', 10);

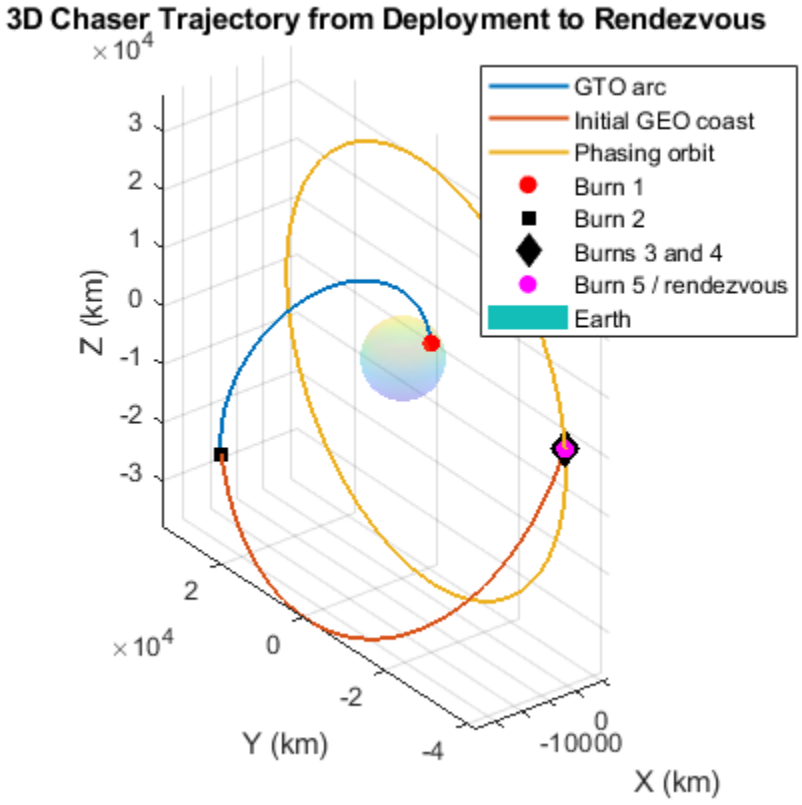
plot3(rendezvous_point(1) / 1000.0, ...
    rendezvous_point(2) / 1000.0, ...
    rendezvous_point(3) / 1000.0, ...
    'mo', 'MarkerFaceColor', 'm');

[earth_x, earth_y, earth_z] = sphere(60);
surf(RE * earth_x / 1000.0, ...
    RE * earth_y / 1000.0, ...
    RE * earth_z / 1000.0, ...
```

```
    'FaceAlpha', 0.20, 'EdgeColor', 'none');

axis equal;
grid on;
xlabel('X (km)');
ylabel('Y (km)');
zlabel('Z (km)');
title('3D Chaser Trajectory from Deployment to Rendezvous');
legend('GTO arc', 'Initial GEO coast', 'Phasing orbit', ...
    'Burn 1', 'Burn 2', 'Burns 3 and 4', ...
    'Burn 5 / rendezvous', 'Earth', ...
    'Location', 'northeast');

drawnow;
snapnow;
```



Local functions.

```
function cost = launch_heading_cost(initial_heading, mission, vehicle)

    launch = simulate_launch(initial_heading, mission, vehicle);

    if ~launch.success
        cost = 1.0e9;
        return;
    end
```

```
    north_hat = local_north_hat( ...
        launch.insertion.latitude, ...
        launch.insertion.longitude);

    east_hat = local_east_hat(launch.insertion.longitude);

    up_hat = local_up_hat( ...
        launch.insertion.latitude, ...
        launch.insertion.longitude);

    velocity_air_relative = ...
        launch.insertion.airspeed * ...
        (cos(launch.insertion.flight_path_angle) * ...
        (cos(launch.insertion.heading) * north_hat + ...
        sin(launch.insertion.heading) * east_hat) + ...
        sin(launch.insertion.flight_path_angle) * up_hat);

    position_surface_fixed = launch.insertion.radius * up_hat;

    earth_rotation_velocity = ...
        cross([0.0; 0.0; mission.OMEGA_EARTH], position_surface_fixed);

    velocity_inertial = velocity_air_relative + earth_rotation_velocity;

    velocity_horizontal = ...
        velocity_inertial - ...
        dot(velocity_inertial, up_hat) * up_hat;

    launch_azimuth = atan2( ...
        dot(velocity_horizontal, east_hat), ...
        dot(velocity_horizontal, north_hat));

    circular_speed = sqrt(mission.MU / norm(position_surface_fixed));

    radial_speed = ...
        dot(position_surface_fixed, velocity_inertial) / ...
        norm(position_surface_fixed);

    azimuth_error = abs(wrap_to_pi( ...
        launch_azimuth - mission.launch_azimuth_target));

    altitude_error = max(0.0, 180e3 - launch.insertion.altitude);

    circular_speed_error = ...
        abs(norm(velocity_inertial) - circular_speed);

    cost = ...
        50.0 * azimuth_error + ...
        1.0e-5 * altitude_error + ...
        1.0e-2 * abs(radial_speed) + ...
        1.0e-2 * circular_speed_error;

end
```

```
function launch = simulate_launch(initial_heading, mission, vehicle)

    solver_options = odeset( ...
        'RelTol', 1.0e-8, ...
        'AbsTol', 1.0e-9, ...
        'MaxStep', 1.0);

    state_0 = [ ...
        mission.RE + mission.launch_altitude; ...
        0.0; ...
        deg2rad(90.0); ...
        vehicle.initial_total_mass; ...
        mission.launch_latitude; ...
        mission.launch_longitude; ...
        initial_heading];

    stage_1_mass_flow = ...
        vehicle.stage_1_thrust / (vehicle.stage_1_isp * mission.G0);

    stage_2_mass_flow = ...
        vehicle.stage_2_thrust / (vehicle.stage_2_isp * mission.G0);

    stage_1_burn_time = ...
        vehicle.stage_1_propellant_mass / stage_1_mass_flow;

    stage_2_burn_time = ...
        vehicle.stage_2_propellant_mass / stage_2_mass_flow;

    insertion_event = @(t, x) insertion_event_function(t, x, mission);
    pitchover_event = @(t, x) pitchover_event_function(t, x, mission);

    phase_1 = ode45( ...
        @(t, x) launch_eom(t, x, mission, vehicle, ...
        vehicle.stage_1_thrust, vehicle.stage_1_isp, true), ...
        [0.0, stage_1_burn_time], state_0, ...
        odeset(solver_options, 'Events', pitchover_event));

    time_1 = phase_1.x.';
    state_1 = phase_1.y.';

    if state_1(end, 1) - mission.RE < mission.pitchover_altitude
        launch = empty_launch_output();
        return;
    end

    state_pit = state_1(end, :).';
    state_pit(3) = mission.pitchover_flight_path_angle;

    remaining_stage_1_propellant = ...
        vehicle.stage_1_propellant_mass - ...
        stage_1_mass_flow * time_1(end);

    remaining_stage_1_burn_time = ...
```

```
    max(0.0, remaining_stage_1_propellant / stage_1_mass_flow);

phase_2 = ode45( ...
    @(t, x) launch_eom(t, x, mission, vehicle, ...
    vehicle.stage_1_thrust, vehicle.stage_1_isp, false), ...
    [time_1(end), time_1(end) + remaining_stage_1_burn_time], ...
    state_pit, ...
    odeset(solver_options, 'Events', insertion_event));

time_2 = phase_2.x.';
state_2 = phase_2.y.';

if state_2(end, 3) <= 0.0 && state_2(end, 1) - mission.RE > 100e3
    [time_all, state_all] = combine_segments({time_1, time_2}, ...
        {state_1, state_2});

    launch = build_launch_output(time_all, state_all, true, mission);
    return;
end

state_sep = state_2(end, :).';
state_sep(4) = state_sep(4) - vehicle.stage_1_structural_mass;

phase_3 = ode45( ...
    @(t, x) launch_eom(t, x, mission, vehicle, ...
    vehicle.stage_2_thrust, vehicle.stage_2_isp, false), ...
    [time_2(end), time_2(end) + stage_2_burn_time], ...
    state_sep, ...
    odeset(solver_options, 'Events', insertion_event));

time_3 = phase_3.x.';
state_3 = phase_3.y.';

if state_3(end, 3) <= 0.0 && state_3(end, 1) - mission.RE > 100e3
    [time_all, state_all] = combine_segments( ...
        {time_1, time_2, time_3}, {state_1, state_2, state_3});

    launch = build_launch_output(time_all, state_all, true, mission);
    return;
end

phase_4 = ode45( ...
    @(t, x) launch_eom(t, x, mission, vehicle, 0.0, 1.0, false), ...
    [time_3(end), time_3(end) + 7200.0], ...
    state_3(end, :).', ...
    odeset(solver_options, 'Events', insertion_event));

time_4 = phase_4.x.';
state_4 = phase_4.y.';

success = state_4(end, 3) <= 0.0 && ...
    state_4(end, 1) - mission.RE > 100e3;

[time_all, state_all] = combine_segments( ...
```

```
        {time_1, time_2, time_3, time_4}, ...
        {state_1, state_2, state_3, state_4});

    launch = build_launch_output(time_all, state_all, success, mission);

end


function state_dot = launch_eom(~, state, mission, vehicle, ...
    thrust, isp, hold_vertical)

    radius = state(1);
    airspeed = state(2);
    flight_path_angle = state(3);
    mass = state(4);
    latitude = state(5);
    heading = state(7);

    altitude = radius - mission.RE;

    density = mission.RHO0 * exp(-max(0.0, altitude) / mission.H_SCALE);
    drag = 0.5 * density * airspeed^2 * vehicle.drag_coef * ...
        vehicle.reference_area;

    gravity = mission.MU / radius^2;

    if thrust > 0.0
        mass_flow = thrust / (isp * mission.G0);
    else
        mass_flow = 0.0;
    end

    if hold_vertical
        radius_dot = airspeed;
        airspeed_dot = (thrust - drag) / mass - gravity;
        flight_path_angle_dot = 0.0;
        latitude_dot = 0.0;
        longitude_dot = 0.0;
        heading_dot = 0.0;
    else
        radius_dot = airspeed * sin(flight_path_angle);

        airspeed_dot = ...
            (thrust - drag) / mass - gravity * sin(flight_path_angle);

        if airspeed < 1.0
            flight_path_angle_dot = 0.0;
            heading_dot = 0.0;
        else
            flight_path_angle_dot = ...
                (airspeed / radius - gravity / airspeed) * ...
                cos(flight_path_angle);

            heading_dot = ...
```

```
                airspeed * cos(flight_path_angle) * sin(heading) * ...
                tan(latitude) / radius;
        end

        latitude_dot = ...
            airspeed * cos(flight_path_angle) * cos(heading) / radius;

        longitude_dot = ...
            airspeed * cos(flight_path_angle) * sin(heading) / ...
            (radius * max(1.0e-8, cos(latitude)));
    end

    mass_dot = -mass_flow;

    state_dot = [ ...
        radius_dot; ...
        airspeed_dot; ...
        flight_path_angle_dot; ...
        mass_dot; ...
        latitude_dot; ...
        longitude_dot; ...
        heading_dot];

end


function [value, isterminal, direction] = ...
    pitchover_event_function(~, state, mission)

    altitude = state(1) - mission.RE;

    value = altitude - mission.pitchover_altitude;
    isterminal = 1;
    direction = 1;

end


function [value, isterminal, direction] = ...
    insertion_event_function(~, state, mission)

    altitude = state(1) - mission.RE;

    if altitude < 100e3
        value = 1.0;
    else
        value = state(3);
    end

    isterminal = 1;
    direction = -1;

end
```

```
function [time_all, state_all] = combine_segments(time_cells, state_cells)

    time_all = [];
    state_all = [];

    for ind = 1:numel(time_cells)
        current_time = time_cells{ind};
        current_state = state_cells{ind};

        if isempty(time_all)
            time_all = current_time;
            state_all = current_state;
        else
            time_all = [time_all; current_time(2:end)];
            state_all = [state_all; current_state(2:end, :)];
        end
    end

end


function launch = build_launch_output( ...
    time_all, state_all, success, mission)

    launch.success = success;

    launch.time = time_all;
    launch.radius = state_all(:, 1);
    launch.airspeed = state_all(:, 2);
    launch.flight_path_angle = state_all(:, 3);
    launch.mass = state_all(:, 4);
    launch.latitude = state_all(:, 5);
    launch.longitude = state_all(:, 6);
    launch.heading = state_all(:, 7);
    launch.altitude = launch.radius - mission.RE;

    launch.insertion.radius = launch.radius(end);
    launch.insertion.altitude = launch.altitude(end);
    launch.insertion.airspeed = launch.airspeed(end);
    launch.insertion.flight_path_angle = launch.flight_path_angle(end);
    launch.insertion.latitude = launch.latitude(end);
    launch.insertion.longitude = launch.longitude(end);
    launch.insertion.heading = launch.heading(end);

end


function launch = empty_launch_output()

    launch.success = false;
    launch.time = 0.0;
    launch.radius = 0.0;
    launch.airspeed = 0.0;
```

```
    launch.flight_path_angle = 0.0;
    launch.mass = 0.0;
    launch.latitude = 0.0;
    launch.longitude = 0.0;
    launch.heading = 0.0;
    launch.altitude = 0.0;
    launch.insertion.radius = 0.0;
    launch.insertion.altitude = 0.0;
    launch.insertion.airspeed = 0.0;
    launch.insertion.flight_path_angle = 0.0;
    launch.insertion.latitude = 0.0;
    launch.insertion.longitude = 0.0;
    launch.insertion.heading = 0.0;

end


function north_hat = local_north_hat(latitude, longitude)

    north_hat = [ ...
        -sin(latitude) * cos(longitude); ...
        -sin(latitude) * sin(longitude); ...
        cos(latitude)];

end


function east_hat = local_east_hat(longitude)

    east_hat = [ ...
        -sin(longitude); ...
        cos(longitude); ...
        0.0];

end


function up_hat = local_up_hat(latitude, longitude)

    up_hat = [ ...
        cos(latitude) * cos(longitude); ...
        cos(latitude) * sin(longitude); ...
        sin(latitude)];

end


function coe = rv_to_coe(position, velocity, mu)

    h_vec = cross(position, velocity);
    h_mag = norm(h_vec);

    r_mag = norm(position);
    v_mag = norm(velocity);
```

```
k_hat = [0.0; 0.0; 1.0];
n_vec = cross(k_hat, h_vec);
n_mag = norm(n_vec);

e_vec = ...
    ((v_mag^2 - mu / r_mag) * position - ...
    dot(position, velocity) * velocity) / mu;

e_mag = norm(e_vec);

specific_energy = v_mag^2 / 2.0 - mu / r_mag;

if abs(e_mag - 1.0) < 1.0e-10
    semi_major_axis = Inf;
else
    semi_major_axis = -mu / (2.0 * specific_energy);
end

inclination = acos(h_vec(3) / h_mag);

if n_mag < 1.0e-12
    raan = 0.0;
else
    raan = atan2(n_vec(2), n_vec(1));
    raan = wrap_to_2pi(raan);
end

if e_mag < 1.0e-10 || n_mag < 1.0e-12
    argument_of_periapsis = 0.0;
else
    argument_of_periapsis = ...
        acos(max(-1.0, min(1.0, dot(n_vec, e_vec) / (n_mag * e_mag))));

    if e_vec(3) < 0.0
        argument_of_periapsis = 2.0 * pi - argument_of_periapsis;
    end
end

if e_mag < 1.0e-10
    true_anomaly = 0.0;
else
    true_anomaly = ...
        acos(max(-1.0, min(1.0, dot(e_vec, position) / ...
        (e_mag * r_mag))));

    if dot(position, velocity) < 0.0
        true_anomaly = 2.0 * pi - true_anomaly;
    end
end

coe.a = semi_major_axis;
coe.e = e_mag;
coe.i = inclination;
```

```
    coe.raan = raan;
    coe.omega = argument_of_periapsis;
    coe.nu = true_anomaly;

end


function h_hat = orbital_plane_normal(raan, inclination)

    h_hat = [ ...
        sin(inclination) * sin(raan); ...
        -sin(inclination) * cos(raan); ...
        cos(inclination)];

    h_hat = h_hat / norm(h_hat);

end


function angle_wrapped = wrap_to_2pi(angle_in)

    angle_wrapped = mod(angle_in, 2.0 * pi);

end


function angle_wrapped = wrap_to_pi(angle_in)

    angle_wrapped = atan2(sin(angle_in), cos(angle_in));

end
```

Discussion of the design reference mission.

Strengths:

This MATLAB script follows the correct order of the required design reference mission. The correct order is the following: launch, GTO transfer, GEO circularization, plane change into the target polar GEO plane, and final two-impulse phasing for rendezvous. The script uses ode45 for launch, enforces the required pitchover setup, computes the launch vehicle's inertial insertion velocity by including the Earth-rotation contribution, and reports all required command window outputs and all nine of the required plots. Also, the target's motion during launch, GTO coast, and initial GEO coast is included when forming the phasing maneuver. This makes the rendezvous timing more physically consistent.

Weaknesses:

The model of the mission is still idealized. This is due to the following reasons: The script's launch uses simplified ballistic assumptions that also includes spherical-Earth gravity, an exponential atmosphere, no winds at all, no Coriolis force in the equations of motion, instantaneous pitchover, instantaneous stage separation, impulsive maneuvers, and Keplerian orbit segments. After deployment, ideal impulsive maneuvers and Keplerian orbit segments are used to model the mission instead of finite-duration burns and a fully continuous propagated trajectory. Also, the plane change is treated like a single instantaneous direction change, and, lastly, the design is not optimized for the minimum propellant usage or minimum total delta-v.

AI Use Statement.

For this assignment's preparation, the author utilized ChatGPT, a language model created by OpenAI. Within this assignment, ChatGPT was used for brainstorming, checking MATLAB code organization, identifying possible errors, polishing code, and suggesting debugging fixes.

Published with MATLAB® R2023b